Chapter 11

Vault: PKI & Automatic Certificate Management

Vault becomes the certificate authority for the entire lab — every TLS certificate auto-issued, auto-renewed, and internally trusted.

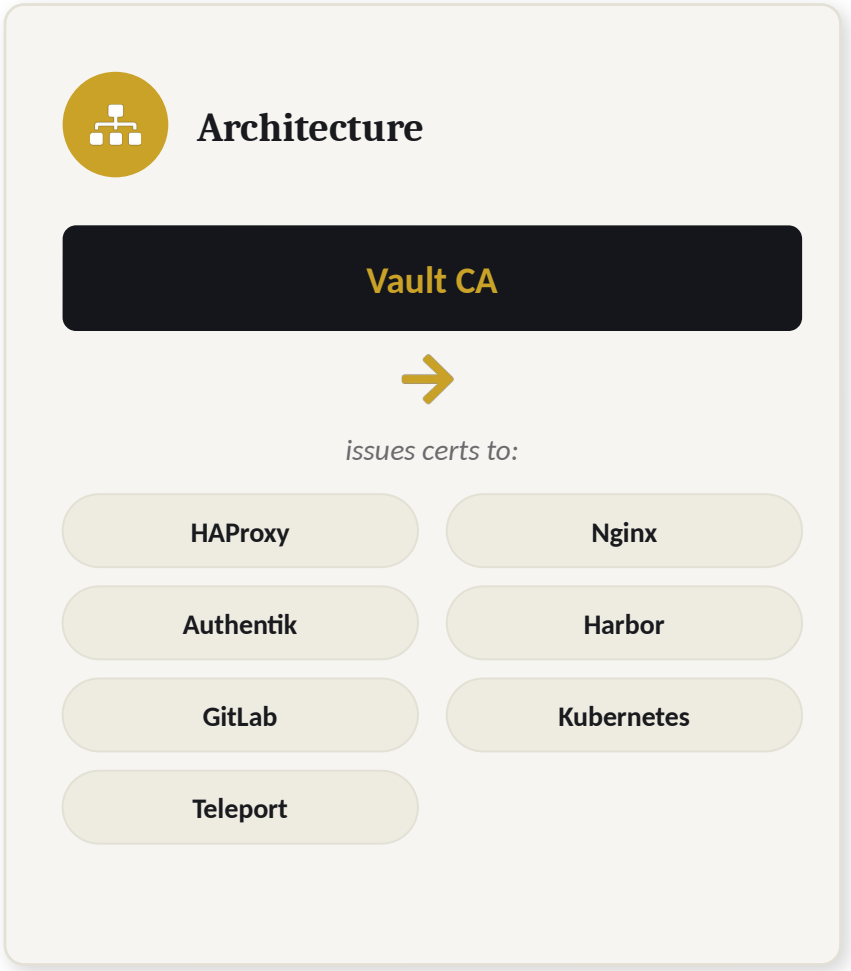
Time estimate: 3–4 hours

Root CA → Intermediate CA → services

What You'll Build in This Chapter

Implement Vault's PKI engine to become the certificate authority (CA) for the entire lab. All TLS certificates are now auto-generated, auto-renewed, and trusted internally.

- Vault is the CA for the devsecops.com domain
- All services use Vault-issued certs — no more self-signed
- Automatic certificate renewal — no manual expiration
- Certificate audit trail — every issuance logged
- Zero certificate warnings in browsers



The Vocabulary of Public Key Infrastructure

Concept	Explanation	Lab Implementation
Root CA	Self-signed, signs other CAs	Vault Root CA
Intermediate CA	Signed by Root, signs end-entity certs	Lab Intermediate CA (*.devsecops.com)
End-Entity Cert	Issued to services (HAProxy, etc.)	Certs for each service
CSR	Certificate Signing Request	Services request certs from Vault
CRL	Certificate Revocation List	List of revoked certs
OCSP	Online Certificate Status Protocol	Real-time cert validation

Without Vault PKI vs. With Vault PKI

WITHOUT VAULT PKI

- Self-signed certs (browser warnings)
- Manual cert generation for each service
- Manual renewal (expiration = service down)
- No audit trail — who created which cert, when?

WITH VAULT PKI

- Trusted internal CA (no warnings)
- Automatic certificate generation
- Automatic renewal (before expiration)
- Complete audit trail

Create the Root CA

Already done in Chapter 10 — shown here for reference.

```
vm# export VAULT_ADDR="https://localhost:8200"
vm# export VAULT_TOKEN="s.xxxxx..."

# Generate root CA
vm# vault write -field=certificate \
    pki/root/generate/internal \
    common_name="DevSecOps Lab Root CA" \
    ttl=87600h > /tmp/root_ca.crt

vm# vault write pki/config/ca \
    pem_bundle=@/tmp/root_ca.crt
```

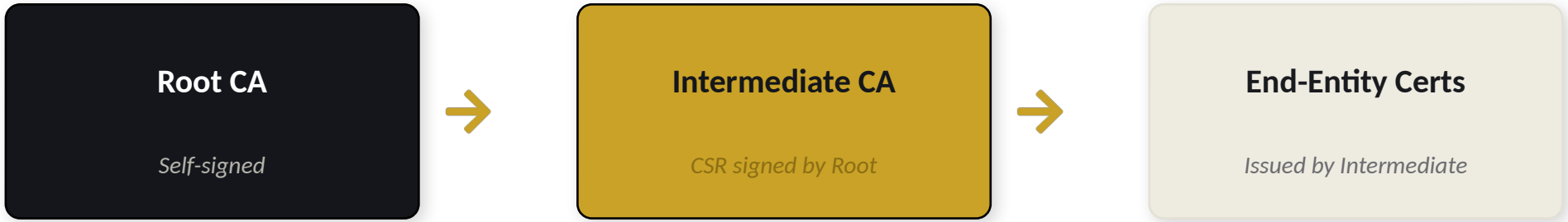


The Trust Anchor

The Root CA is self-signed and never used directly to issue service certificates. A 10-year TTL (87600h) reflects how rarely it should ever need to change.

Kept offline / cold where possible — compromise here compromises everything downstream.

Create the Intermediate CA




```
# 1. Enable intermediate PKI engine
vm# vault secrets enable -path=pki_int pki
vm# vault secrets tune -max-lease-ttl=43800h pki_int

# 2. Generate CSR, then sign it with the Root CA
vm# vault write -field=csr pki_int/intermediate/generate/csr \
    common_name="Lab Intermediate CA" > /tmp/pki_intermediate.csr
vm# vault write -field=certificate pki/root/sign-intermediate \
    csr=@/tmp/pki_intermediate.csr ttl=43800h > /tmp/pki_intermediate.crt

# 3. Set the signed cert back onto the intermediate engine
vm# vault write pki_int/intermediate/set-signed certificate=@/tmp/pki_intermediate.crt
```

Certificate Roles Define What Can Be Issued

Each role constrains domain, TTL, and usage — services can only request certs their role allows.

`pki_int/roles/lab`	***.devsecops.com**	*720h max*	`allowed_domains=devsecops.com, allow_subdomains=true`
`pki_int/roles/k8s`	***.cluster.local**	*720h max*	`allowed_domains=cluster.local, allow_subdomains=true`
`pki_int/roles/services`	**Named SANs only**	*720h max*	`vault, gitlab, harbor .devsecops.com — allow_subdomains=false`

Issue a Wildcard Certificate to HAProxy



Why combine cert + key into one PEM?

HAProxy expects a single file containing both the certificate chain and the private key for each frontend binding — hence the cat step before deployment.

Confirm the Certificate Vault Issued

```
$ openssl x509 -in /tmp/wildcard.crt -text -noout | head -30
```

```
Certificate:
    Data:
        Version: 3 (0x2)
        Serial Number: 01:23:45:...
        Signature Algorithm: sha256WithRSAEncryption
        Issuer: CN = Lab Intermediate CA
        Subject: CN = *.devsecops.com
        Subject Public Key Info:
        X509v3 Subject Alternative Name:
            DNS:*.devsecops.com
            DNS:devsecops.com
```

What to check

Issuer

Must read "Lab Intermediate CA" — proves the chain, not a self-signed fallback

Subject

Matches the requested common_name exactly

SANs

Wildcard + bare domain both present so both resolve without warnings

Refresh Certificates Before They Expire

Check age

Read cert's enddate, compute days remaining

→

≤ 7 days left?

Threshold trips renewal — else exit quietly

→

Request new cert

POST to Vault's pki_int/issue/lab endpoint

→

Validate & swap

openssl checks new file, then atomic mv

→

Reload service

systemctl reload haproxy — zero downtime

Runs daily via cron

```
0 2 * * * root /usr/local/bin/refresh-vault-cert.sh
  >> /var/log/vault-cert-renewal.log 2>&1
```

30-day certs, renewed at day 23. A 7-day buffer absorbs Vault downtime, network blips, or a missed cron run without ever risking an expired cert in production.

Every Service Gets Its Own Certificate

Authentik

`CN: auth.devsecops.com`

/etc/authentik/certs/tls.crt + tls.key

Vault (itself!)

`CN: vault.devsecops.com`

Replaces the self-signed cert from Ch. 10

Kubernetes API

`CN: kubernetes.default.svc`

Role: k8s — includes cluster.local SANs

etcd

`CN: etcd.local`

SANs: etcd, localhost, 127.0.0.1

Configure the Certificate Revocation List

```
# Enable CRL endpoint
vm# vault write pki_int/config/crl \
    disable=false

# Set CRL distribution point
vm# vault write pki_int/config/urls \
    crl_distribution_points=\
    "https://vault.devsecops.com/v1/pki_int/crl/pem"

# View the CRL
vm# curl -s https://10.0.2.13:8200/v1/pki_int/crl/pem \
    | openssl crl -inform PEM -text -noout | head -20
```



Why revoke at all?

A stolen private key, a decommissioned service, or a cert issued with the wrong SANs all need a way to be invalidated before their TTL naturally expires.

Clients checking the CRL distribution point will reject a revoked cert even if it hasn't expired.

Every Certificate Issuance Is Logged

```
$ sudo tail -100 /var/log/vault/audit.log | grep "pki_int/issue" | jq -r
'.request.path, .auth.metadata.user_id, .request.data'
```



Who issued it

The user or app token that made the request



When

Exact timestamp of issuance



What

Common Name and SANs on the issued certificate



Result

Success or failure of the request

check-cert-expiry.sh — Catch Problems Before They're Outages

```
DAYS_WARNING=14

# List certs, check each one's expiry
for CERT_ID in $CERTS; do
  EXPIRY=$(... .data.expiration_time)
  CN=$(... .data.certificate_details.common_name)
  DAYS_LEFT=$(( (EXPIRY_EPOCH - NOW_EPOCH) / 86400 ))

  if [ "$DAYS_LEFT" -le "$DAYS_WARNING" ]; then
    echo "⚠️  ALERT: $CN expires in $DAYS_LEFT days"
  elif [ "$DAYS_LEFT" -le 0 ]; then
    echo "🚨 ERROR: $CN has expired!"
  else
    echo "✅ OK: $CN expires in $DAYS_LEFT days"
  fi
done
```



Three states, one glance

✅ **OK**

More than 14 days remaining — no action needed

⚠️ **ALERT**

14 days or fewer left — renewal window is open

🚨 **ERROR**

Already expired — treat as an active incident

Fold Certificate Issuance Into Playbooks

```
# roles/haproxy/tasks/main.yml
- name: Request certificate from Vault
  uri:
    url: "https://{{ vault_address }}/v1/pki_int/issue/lab"
    method: POST
    headers:
      X-Vault-Token: "{{ vault_token }}"
    body_format: json
    body:
      common_name: "*.devsecops.com"
      ttl: "30d"
  register: vault_cert

- name: Deploy certificate
  copy:
    content: |
      {{ vault_cert.json.data.certificate }}
      {{ vault_cert.json.data.private_key }}
    dest: /etc/haproxy/certs/devsecops.com.pem
    mode: '0600'
  notify: reload haproxy
```



One notify, one reload

The same handler pattern from Chapter 09 applies here: the uri task requests a fresh cert, the copy task writes it, and the handler reloads HAProxy only if the file actually changed.

This is how certificate issuance becomes part of every VM's regular configuration run — not a one-off manual step.

Three Problems You'll Likely Hit

Certificate validation failed

The issuer isn't trusted yet

cp root_ca.crt to ca-certificates, then update-ca-certificates (or VAULT_SKIP_VERIFY for testing only)

Certificate renewal fails

Vault connectivity or auth issue

curl the roles endpoint directly with the token; check audit.log for the specific error

Wrong CN or SANs on cert

Issued with incorrect parameters

Inspect with openssl x509 -text; revoke the bad serial via pki_int/revoke; re-issue correctly

Chapter Summary — What We Built

What We Built	Purpose
Root CA	Trust anchor for entire lab
Intermediate CA	Issues end-entity certs
Certificate roles	Define what certs can be issued
Service certificates	Trusted certs for all services
Auto-renewal	Certificates refresh before expiry
CRL	Certificate revocation
Audit logging	Certificate issuance tracked

Key Takeaway

No more manual certificate management.

- Trusted internal CA — no browser warnings
- Automatic renewal — running 30+ days
- Expiration is a non-issue, not a failure point
- Complete audit trail of every issuance

PHASE 3 CONTINUES

Vault Now Secures Every Certificate. Next: Securing Every Access.

Chapter 12 → Teleport: Zero Trust Access & Session Recording

- Replace direct SSH with Teleport proxy
- Every access recorded — session video
- RBAC — role-based access control
- Audit log of every action
- Multi-service access (SSH, DB, Kubernetes)